

User and Permission Management

User Type	Description
Root (Superuser)	Full system control. Can install software, change config files, and delete anything. Powerful but risky.
Regular User	Limited access. Can create files, run applications, but not modify system-level settings.
Sudo User	Regular user with temporary admin rights via the sudo command. Common in modern systems.
System/Service Account	Non-human accounts used by services (e.g., mysql, nginx). Limited privileges.
Guest User	Temporary users with minimal privileges. Changes are not saved after logout.

UID 0
Reserved for the
root (superuser)

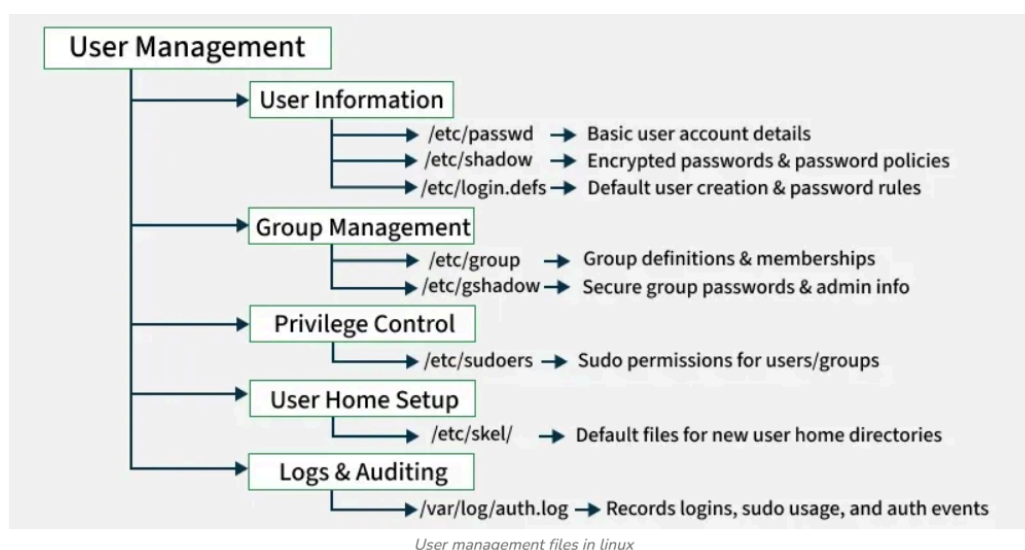
UID 1-999
System accounts
used by services

UID 1000+
Regular users

Human users : `awk -F: '$3 >= 1000 { print $1 }' /etc/passwd`

Linux systems typically support up to 60,000 users

A **user group** is a collection of users. If you give permission to a group, all users in that group get the same access. This makes it easier to manage file and system permissions for many users at once.



- On Debian-based systems (Kali, Ubuntu), authentication logs are in `/var/log/auth.log`
- On RHEL/CentOS/Fedora systems, they are stored in `/var/log/secure`

- On modern systemd systems, logs can also be viewed using *journalctl*

```
hehkek@hehkek:~$ awk -F':' '{ print $1}' /etc/passwd
root
daemon
bin
sys
sync
games
man
lp
mail
news
uucp
proxy
www-data
backup
list
irc
gnats
nobody
systemd-network
systemd-resolve
messagebus
systemd-timesync
syslog
_apt
tss
uuidd
tcpdump
sshd
pollinate
landscape
fwupd-refresh
_chrony
hehkek
lxd
snort
```

/etc/passwd : This file stores **all users on the system**.

username : password : UID : GID : comment : home : shell (line format in etc/password)

awk -F ':' '{ print \$1, \$3 }' /etc/passwd -to print username + uid

awk is a powerful text-processing command in Linux used to analyze, filter, and manipulate structured data such as logs, CSV files, and command output. It works by scanning input line by line and performing actions based on patterns and fields.

-F ':' Field separator = :

- So awk splits each line like: root | x | 0 | 0 | root | /root | /bin/bash

To add a user : `useradd username per`

`Sudo adduser name`

```
hehkek@hehkek:~$ sudo adduser exam_1
Adding user `exam_1' ...
Adding new group `exam_1' (1001) ...
Adding new user `exam_1' (1001) with group `exam_1' ...
Creating home directory `/home/exam_1' ...
Copying files from `/etc/skel' ...
New password:
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N] y
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for exam_1
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
```

```

hekhek@hekhek:~$ who
hekhek  ttyS0      2026-03-28 05:44
hekhek  pts/0      2026-03-29 18:04 (103.70.197.3)
hekhek@hekhek:~$ id
uid=1000(hkekhek) gid=1000(hkekhek) groups=1000(hkekhek),4(adm),20(dialout),24(cdrom),25(floppy),26(lxd),27(sudo)
hekhek@hekhek:~$ finger
Command 'finger' not found, but can be installed with:
sudo apt install finger
hekhek@hekhek:~$ sudo apt install finger

```

Who : list all logged in users-list all sessions [w - for more detailed who]

Id : get user and grp id

Finger : logged in user information

```

hekhek@hekhek:~$ finger
Login      Name      Tty      Idle   Login Time   Office      Office Phone
hekhek     Ubuntu    *ttyS0    1d     Mar 28 05:44
hekhek     Ubuntu    pts/0     Mar 29 18:04 (103.70.197.3)

```

Two active login sessions of the SAME user (hekhek) ;

*ttyS0

- tty = terminal
- ttyS0 = serial console
- * = currently active session

pts/0

- pts = pseudo terminal (used by SSH)
- Each SSH login → new pts

```

hekhek@hekhek:~$ tty
/dev/pts/0
hekhek@hekhek:~$ pts
Command 'pts' not found, but can be installed with:
sudo apt install openafs-client
hekhek@hekhek:~$ w
 18:23:27 up 1 day, 12:39,  2 users,  load average: 0.03, 0.04, 0.00
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
hekhek    ttyS0    -               Sat05    36:15m 0.41s  0.38s  -bash
hekhek    pts/0    103.70.197.3    18:04    0.00s  0.05s  0.00s  w

```

tty = “which terminal am I using right now?”

1. ttyS0 :Physical / virtual console ; Example: Azure serial console
2. pts/0, pts/1, etc. SSH terminals , Every SSH login → new pts/X

pskill = Process Kill

pskill -t pts/0 ; Kill all processes running on terminal **pts/0**

"Kill everything running on my current terminal" but Linux avoids breaking your current session in that way (or only kills child processes)

```
hekhek@hekhek:~$ pskill -t pts/0
hekhek@hekhek:~$ finger
```

Login	Name	Tty	Idle	Login Time	Office	Office Phone
hekhek	Ubuntu	*ttyS0	1d	Mar 28 05:44		
hekhek	Ubuntu	pts/0		Mar 29 18:04	(103.70.197.3)	

Now,

```
hekhek@hekhek:~$ sudo pskill -t ttyS0
hekhek@hekhek:~$ finger
```

Login	Name	Tty	Idle	Login Time	Office	Office Phone
hekhek	Ubuntu	pts/0		Mar 29 18:04	(103.70.197.3)	

`useradd new_username`

- Creates user account only
 - No home directory
 - No password set
 - Default shell = **sh**
- `useradd -d /path/to/dir new_username` ; custom home directory
→ `useradd -m new_username` ; -m creates /home/username
→ `useradd -s /bin/bash new_username` ; set default shell as bash

- -m → Create home directory
- -d → Set custom home directory
- -s → Set default shell
- -g → Set primary group
- -G → Add to supplementary groups
- -u → Set user ID (UID)

Eg : `useradd -m -s /bin/bash -g dev -G sudo user1`

`userdel` - delete a user ; -f force removal (even if logged in) ; -r (removes files and stuffs also, else no removal of files)

usermod - modify a user acc ;

- `-l` → Change username
- `-d` → Change home directory
- `-m` → Move contents to new home
- `-s` → Change shell
- `-g` → Change primary group
- `-G` → Set supplementary groups
- `-aG` → Append to groups (important-without a, -G will overwrite existing groups)

Eg : `usermod -s /bin/zsh user1`

`usermod -aG sudo user1`

`usermod -l newname oldname`

`passwd username` → Set/change password

- `-l` → Lock account
- `-u` → Unlock account
- `-d` → Delete password (no password login)
- `-e` → Expire password (force change on next login)

```
hekh@hekh:~$ sudo usermod -aG sudo examadmin
hekh@hekh:~$ sudo groupadd auditgroup
hekh@hekh:~$ sudo usermod -aG auditgroup examaudit
hekh@hekh:~$ sudo chmod 750 /hom/exam_1
chmod: cannot access '/hom/exam_1': No such file or directory
hekh@hekh:~$ sudo chmod 750 /home/exam_1
hekh@hekh:~$ sudo chmod 750 /home/exam_2
hekh@hekh:~$ sudo chmod 750 /home/exam_3
hekh@hekh:~$ sudo chgrp auditgroup /home/exam_1
hekh@hekh:~$ sudo chgrp auditgroup /home/exam_2
hekh@hekh:~$ sudo chgrp auditgroup /home/exam_3
hekh@hekh:~$ awk -F ':' '$3>=1000{ print $1}' /etc/passwd
nobody
hekh
exam_1
exam_2
exam_3
examadmin
examaudit
```

`sudo usermod -aG sudo examadmin` ; add examadmin to sudo group

`sudo groupadd auditgroup` ; new group auditgroup is created

`sudo usermod -aG auditgroup examaudit` ; examaudit is added to that group

`sudo chmod 750 /home/exam_1` ; owner=7,group=5,others=0

`sudo chgrp auditgroup /home/exam_1` ; changed group of these directories to auditgroup
;This does **NOT change user's groups**

- **Owner (u)** → user who owns file
- **Group (g)** → group assigned
- **Others (o)** → everyone else

So now, examaudit: User with read-only access to all user home directories.

<code>groupadd</code>	Creates new groups
<code>groupdel</code>	Delete a group
<code>groupmod</code>	Modify group properties

`sudo apt install quota` [Limits how much disk space a user can use]

`/etc/fstab` is a configuration file that tells the system how to mount filesystems during boot. Each line represents one filesystem and contains instructions for:

- Which disk partition to use
- Where to attach it in the directory tree
- What filesystem type it is
- What options to apply
- Whether to check or back it up

`UUID=326bf812-1a76-40fe-8839-c1847638810f / ext4 discard,errors=remount-ro 0 1`

discard

- Used for SSDs.
- When files are deleted, the OS informs the SSD that blocks are free.
- This helps maintain SSD performance and lifespan.

errors=remount-ro

- If filesystem errors occur, it remounts as read-only. This prevents further corruption.
- You can still read files but cannot write.

ext4 (fourth extended filesystem) is a [journaling file system](#) for [Linux](#), developed as the successor to [ext3](#).

0 = Dump flag.

- Used by an old backup tool ([dump](#)).
- 0 means do not include this filesystem in backups.

1 = Filesystem check order (fsck).

- 1 means this filesystem is checked first during boot.
- Root filesystem is always given priority.

Now, i added `discard,errors=remount-ro,usrquota`

```
UUID=88AF-3759 /boot/efi vfat umask=0077 0 1
```

/boot/efi = Mount point.(for first one /)

- This directory contains bootloader files.
- Required for UEFI systems.

Vfat =Filesystem type.

- EFI partitions must use FAT (vfat).

umask=0077 = Permission mask.

- Only root can access files.
- All other users are denied access.

When you edit `/etc/fstab`, nothing happens immediately. The kernel only reads that file during the boot process.

- **The Command:** `-o remount` tells the system: "Don't unmount the drive (which would crash the OS), just refresh the settings for the already active mount point (/)."
- **The Result:** The kernel now sees the `usrquota` flag and prepares the filesystem to start tracking file ownership for quota purposes.

`sudo quotacheck -cum /`

- `-c` → create quota files
- `-u` → user quota
- `-m` → don't remount read-only

- **-c (Create):** Creates the storage files (like `aquota.user`) at the root of the drive. These act as the "ledger" or "database."
- **-u (User):** Tells the tool to scan for user-based quotas (as opposed to group quotas).
- **-m (No Remount):** Normally, `quotacheck` wants to make the drive "Read-Only" so no files change while it's counting. `-m` forces it to run on a "live" system.

The Result: The system crawls every single folder, identifies who owns what, and calculates the total blocks currently used.

```
hekhek@hekhek:~$ sudo quotaon /
quotaon: using //aquota.user on /dev/root [/]: No such process
quotaon: Quota format not supported in kernel.
```

```
hekhek@hekhek:~$ lsmod | grep quota
hekhek@hekhek:~$ sudo modprobe quota_v2
modprobe: FATAL: Module quota_v2 not found in directory /lib/modules/6.8.0-1044-azur
re
hekhek@hekhek:~$ mount | grep " / "
/dev/sda1 on / type ext4 (rw,relatime,discard,quota,usrquota,errors=remount-ro)
hekhek@hekhek:~$ ls -l /aquota.*
-rw----- 1 root root 7168 Mar 30 08:19 /aquota.user
```

Permissions and quotas differ fundamentally in how they are implemented and enforced in a Linux system.

Permissions are stored as metadata within each file's inode, including the owner, group, and access bits. When a process attempts to access a file, the kernel performs a simple check against these permission bits to determine whether the operation is allowed. This check is local to the file and does not require any global state or tracking. Since permission handling is an essential part of basic filesystem operation, it is always supported by the kernel and does not require additional modules or features.

Quotas, on the other hand, are designed to limit the total disk usage of users or groups across the entire filesystem. This requires maintaining a global record of how much space each user is consuming. The system builds and updates this information using separate quota database files such as `aquota.user`. Every time a file is created, modified, or deleted, the kernel must update these usage records and verify them against predefined limits before allowing further writes.

Because quota enforcement involves continuous tracking, global accounting, and interception of write operations, it requires explicit support within the kernel. If the kernel lacks the necessary quota modules or format support, the system may still create quota files and track usage, but it will not be able to enforce limits. This is why quota functionality depends on kernel capabilities, whereas permissions operate independently as part of the core filesystem design.

enabled `usrquota` in `/etc/fstab`, remounted the filesystem, created the quota database (`/aquota.user`) using `quotacheck`, and verified that the filesystem was mounted with quota options. However, when you tried to enable quotas using `quotaon`, it failed because your system is running an Azure kernel that does not include the required quota support modules. This is why `modprobe quota_v2` also failed—those modules are simply not present in your kernel. So, even though your configuration and commands were correct at the filesystem level, the kernel itself lacks the capability to enforce quotas, which is why the process did not work.

Solution : install a full generic kernel ; current system = partially supports quota
,Generic kernel = fully supports quota;

```
sudo apt update
```

```
sudo apt install linux-image-generic linux-headers-generic
```

sudo quotaon / : This is the "On/Off" switch.

- **Concept:** Even after scanning, the kernel isn't stopping anyone yet. `quotaon` activates the enforcement engine. From this microsecond forward, every time a user tries to save a file, the kernel checks the `aquota.user` file to see if they have "permission" (space) to do so.

The Policy: `sudo edquota exam_1` This opens a text editor (usually `vi` or `nano`) showing the current limits for that specific user. **Soft Limit:** The "Warning" zone. The user can go over this for a "grace period" (usually 7 days), but they get a warning. **Hard Limit:** The "Brick Wall." The user can **never** exceed this number. **Blocks:** This is measured in **1KB increments**. So, if you want a 100MB limit, you'd enter `100000`.

The Audit: `quota -u exam_1` This is the verification tool. It provides a report showing:

- How much the user is using right now.
- Where their "Soft" and "Hard" walls are.
- If they are currently in a "Grace" period (over the soft limit).

(Unfortunately i cant do this!)

```
#!/bin/bash
BACKUP_DIR="/secure_backup"
DATE=$(date +%F)

# Ensure the backup folder exists
mkdir -p $BACKUP_DIR

# Compress all exam user folders
tar -czf $BACKUP_DIR/exam_backup_$DATE.tar.gz /home/exam_*
```

In shell scripting, the shebang (`#!/bin/bash`) specifies which interpreter should run a script.

Format of shebang ; `#!/path/to/interpreter`

`date +%F` produces a standard "Full Date" format: `YYYY-MM-DD`.

The `$(...)` syntax is **Command Substitution**. It runs the command inside and assigns the output to the variable.

The -p flag (Parents): This is a safety "Idempotent" flag. (so that script doesn't crash at first)

- If the folder `/secure_backup` already exists, `mkdir` usually throws an error. `-p` tells it: "If it exists, do nothing and don't complain. If it doesn't exist, create it (including any parent folders)."

tar: Short for "Tape Archiver." Historically used for backups on magnetic tapes; now the standard for bundling files.

-c (Create): Tells tar to create a new archive file.

-z (gzip): This is crucial. It tells tar to filter the archive through **gzip** compression. Without this, the file would be huge.

-f (File): Tells tar that the next argument is the name of the archive file. **Note:** `-f` must always be the last flag in the cluster (e.g., `-czf` is fine, but `-cfz` will fail because it looks for a file named `z`).

/home/exam_*: The asterisk is a **wildcard**. The shell expands this to every folder starting with `exam_`.

```

hekhek@hekhek:~$ sudo nano /usr/local/bin/backup_exam.sh
hekhek@hekhek:~$ sudo mkdir -p /secure_backup
hekhek@hekhek:~$ sudo chown examadmin:examadmin /secure_backup
hekhek@hekhek:~$ sudo chmod 700 /secure_backup
hekhek@hekhek:~$ sudo chown examadmin:examadmin /usr/local/bin/backup_exam.sh
hekhek@hekhek:~$ sudo chmod 700 /usr/local/bin/backup_exam.sh
hekhek@hekhek:~$ sudo su - examadmin
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

examadmin@hekhek:~$
examadmin@hekhek:~$ /usr/local/bin/backup_exam.sh
tar: Removing leading '/' from member names
tar: /home/exam_1: Cannot open: Permission denied
tar: /home/exam_2: Cannot open: Permission denied
tar: /home/exam_3: Cannot open: Permission denied
tar: Exiting with failure status due to previous errors
examadmin@hekhek:~$ sudo /usr/local/bin/backup_exam.sh
[sudo] password for examadmin:
tar: Removing leading '/' from member names
tar: Removing leading '/' from hard link targets
examadmin@hekhek:~$ ls -lh /secure_backup
total 4.0K
-rw-rw-r-- 1 examadmin examadmin 2.6K Mar 30 08:58 exam_backup_2026-03-30.tar.gz

```

sudo nano ...: You created the script file.

sudo mkdir -p /secure_backup: You created the destination folder.

sudo chown examadmin:examadmin ...: You gave ownership of both the script and the folder to the user examadmin.

sudo chmod 700 ...: You locked them down so only examadmin can see or run them.

- **Removing leading '/' from member names** : This is a **safety feature** of **tar** . It changes **/home/exam_1** to **home/exam_1** (relative path) inside the zip file. This is done so that if you restore the backup later, it doesn't accidentally overwrite your actual system files; it puts them in the current folder instead.
- **ls -lh /secure_backup** : You verified the result.
 - **2.6K** : The size of the backup (very small because your exam users likely have empty folders right now).
 - **exam_backup_2026-03-30.tar.gz** : The filename correctly used your **\$(date +%F)** variable.

Crontab -e : A *crontab* file contains instructions to the *cron*(8) daemon of the general form: ``run this command at this time on this date''. Each user has their own crontab, and commands in any given crontab will be executed as the user who owns the crontab.

```
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 2 * * * /usr/local/bin/backup_exam.sh
```

The Crontab format follows: Minute Hour Day-of-Month Month Day-of-Week

0 2 * * *: This means at the 0th minute of the 2nd hour (2:00 AM) every single day.

To ensure this works automatically while you sleep, you should move the task to the **Root Crontab**. This gives the script "Superuser" powers without needing you to type a password. (currently i did `crontab -e`, and edited that; now i have to delete it from here and do the same for root crontab -e)

A script that runs silently is scary. You can redirect errors to a log file so `examadmin` can check if it worked.

Bash

```
tar -czf $BACKUP_DIR/backup.tar.gz /home/exam_* >> /var/log/backup.log 2>&1
```

>> /var/log/backup.log: Instead of printing "Success" to a screen no one is watching, it saves it to a file.

2>&1: This is a classic Linux trick. 2 is **Standard Error** and 1 is **Standard Output**. This tells Linux: "Send the error messages to the same place as the success messages." If the backup fails, the reason will be written in `/var/log/backup.log`.

```
#!/bin/bash
BACKUP_DIR="/secure_backup"
DATE=$(date +%F)

# Ensure the backup folder exists
mkdir -p $BACKUP_DIR

# Compress all exam user folders
tar -czf $BACKUP_DIR/exam_backup_$DATE.tar.gz /home/exam_*
tar -czf $BACKUP_DIR/backup.tar.gz /home/exam_* >> /var/log/backup.log 2>&1
find $BACKUP_DIR -name "*.tar.gz" -mtime +7 -delete
```

find \$BACKUP_DIR: Look inside the backup folder.

-name "*.tar.gz": Only look for our backup files (don't delete other things!).

-mtime +7: Find files that were modified **more than 7 days ago**.

-delete: Remove them.